INDEX

NAME: Kiruthick.sm STD.: CSE SEC.: _____ ROLL NO.: 263 SUB.: Computer Networks

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
01.	20.07.26	Study of various commands used in Linux and windows		9/8/26 (9)
02.	27.07.26	Study of different types of network cables.		4/8/26 (9)
03.	04.08.26	Cisco Packet Tracer.		15/8/26 (9)
04.	10.08.26	Setup & configure a LAN using a switch and ethernet cables in your lab.	(9)	13/8
05.	18.08.26	Wire Shark	(9)	18/8
06.	07.09.26.	Error detection & Error correction.	(9)	7/9
07.	08/09/26	Flow control using Sliding window protocol.	(9)	5/9
08.	22.09.26	NMAP to Discover live hosts using Nmap scans on the tryhackme platform.	(9)	22/9
09.	28.09.26	Implementation of Subnetting in cisco packet Tracer Simulator.	(9)	28/9

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
12.	29.9.26.	End to End communication at Transport layer.	9.	29/9
13.	29.9.26.	Implementation your own ping command.	10	29/9
14.	01.10.26	Packet Shifting using Raw Socket.	10	01/10/26

Exp : No : 01 Date: 20-07-26

Study of various Network commands used in Linux and Windows.

Aim:

To study various network commands used in windows and Linux

Windows commands:

1. arp -a — Display the IP addres and MAC addres of devices in the Network.
2. hostname — Displays the name of the computer
3. ipconfig /all — Shows complete network configuration (IP, DNS, MAC, etc)
4. Nbstat -a — Displays NetBios information of a computer.
5. net stat — Displays active network connection and listening ports.
6. netstat -r — Displays the routing table.
7. NS lookup — D. finds the IP addres of a website using DNS.
8. pathping <host> — Display the path to a destination and checks packet loss

9. ping <host/IP> – Tests connectivity between 2 devices.

10. route – Display or modifies the IP routing table.

Linux commands.

1. ip address show – Displays the IP addres of all network interface.

2. ip link set eth0 up – Enables the network interface

3. ip link set eth0 down – Disable the network interface

4. ip route add <network> – Adds a route to a network.

5. ip route get <IP> – Displays the route to a destination IP address.

6. ifconfig – Displays network interface details.

7. mtr <host> – Shows the route to a destination and network statistics.

8. ip route add default via <gateway> – Adds the default gateway.

9. ping <host> – checks whether a remote host is reachable.

10. nmcli connection show – Displays all network connection profiles.

Result:

Thus, the various networking commands used in windows and linux were studied and executed successfully.

Windows commands:

1. arp -a

ARP is short form of Address Resolution Protocol.

O/p: Interface 10.16.56.50 --- 0x9

Internet Address	physical Address	Type
192.168.1.1	00-1A-2B-3C-4D-5E	Dynamic
192-168.1.20	08-00-27-AB-CD-EF	Dynamic

2. hostname

To display the computer name.

O/p: DESKTOP-PC01.

3. ipconfig/all

To display complete network configuration.

O/p: IPv4 Address : 192.168.1.1.0
Subnet Mask : 225.255.255.0
Gateway : 192.168.1.1
DNS Server : 8.8.8.8.

4. nbtstat -a

To display NetBIOS information.

Sample o/p : NetBIOS Remote Machine Name Table

Name	TYPE
DESKTOP-PC01	UNIQUE
WORKGROUP	GROUP

5. netstat

To display active network connection and listening ports.

Sample output:

Proto	Local Address	Foreign Address	State
TCP	192.168.1.10:5050	142.250.183.36:443	Established
TCP	0.0.0.0:135	0.0.0.0:0	Listening

6. netstat -r

To display routing Table.

Sample o/p:

Network Destination	Gateway	Interface
0.0.0.0	192.168.1.1	192.168.1.10

7. nslookup www.google.com

To find the IP address of a domain name

Sample o/p:

Server : dns.google
Name : www.google.com
Addres : 142.250.183.36.

8. pathping google.com

To display the network path & packet loss

Sample o/p:

Tracing route to google.com
Hop 1 192.168.1.1
Hop 2 10.10.10.1
Hop 3 google.com
computing statistics.....

9. Ping google.com

To test network connectivity with another host.

Sample o/p:

ping ping google.com [142.250.183.36]
Reply from 142.250.183.36:
Bytes = 32
Time = 15ms
TTL = 118

10. route print.

To display the IP routing table

Sample output:

Network Destination	Netmask	Gateway
0.0.0.0	0.0.0.0	192.168.1.1
192.168.1.0	255.255.255.0	On-link

Linux command.

1. IP command - ip address show.

To display the IP addresses of all network interface.

Sample o/p:

2: eth0
inet 192.168.1.10/24
inet 6 fe80::a00:27ff:fe4e:6691/64

2. ip address add 192.168.1.100/24 dev eth0

To assign an IP address to network interface.

Sample o/p:

IP address added succesfully.

3. ip addres del 192.168.1.100/24 dev eth0.

To remove an IP address from a network interface.

Sample o/p:

IP address removed successfully.

4. ip link set eth0 up

To enable a network interface

Sample o/p: eth0: state UP

5. ip link set eth0 down

To disable a network interface

Sample o/p: eth0: state down.

6. ip route add default via 192.168.1.1. dev eth0

To add the default gateway.

Sample o/p: Default route added successfully.

7. ip route add 192.168.2.0/24 via 192.168.1.1

To add a route to another network

Sample o/p: Route added successfully

8. ip route delete 192.168.2.0/24 via 192.168.1.1

To delete a route from the routing table

Sample o/p: Route deleted successfully

9. ip route get 10.10.1.4

To display the route to a destination

Sample o/p : 10.10.1.4 via 192.168.1.1 dev etho

10. ifconfig

To display network interface network.

sample o/p :

etho : flags = 4163 < UP, BROADCAST, RUNNIN>
inet 192.168.1.10
netmask 255.255.255.0.

11. mtr google.com

To display the network path & statistics

Sample o/p :

HOST	Loss%	Avg
192.168.1.1	0.0%	1.2ms
google.com	0.0%	18.5 ms

12. tcpdump -D

To display available network interface

Sample o/p : 1. etho
2. lo.

13. tcpdump -i eth0

To capture network packets

Sample o/p : IP 192.168.1.10 > 8.8.8.8 : ICMP Echo Request

14. ping google.com

To test network connectivity.

Sample o/p : 64 bytes from google.com:
time = 12.4 ms
ttl = 118-

15. nmcli connection show

To display network connection profiles

Sample o/p:

NAME : Wired connection 1
UUID : a5ab6490-cc20-3668-8158-08149273f375
TYPE : ethernet
DEVIC : eth0.

28/7/26

Result:

Thus, the various networking commands used in windows & Linux were studied and executed successfully.

EXP :NO: 02 Date: 27.07.26

Study of different types of Network cable

Aim:

To Study different types of Network cable.

a) Understand different types of network cables.

1. Unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) cable.
3. Coaxial Cable.
4. Fibre optic cable.

Cabel type	category	Max. Data Transmission	Advantage / Disadvantage	Application / Use
UTP	category 3	10bps	Advantage: * cheaper in cost * Easy to install as they have a smaller overall diameter. Disadvantage: * More prone to (EMI) Electromagnetic interference & noise.	* 10 Base T-ethernet * Fast ethernet, Gigabit ethernet * Fast ethernet, Gigabit ethernet
	Category 5	upto 100 Mbps		
	category 5e	1Gbps		
STP	category 6, 6a	10Gbps	Advantage: * Shielded * Faster than UTP * less susceptible to noise & interference	* Gigabit ethernet, 10G Ethernet (55m) widely used in data centres.

SSTP	category 7	10Gbps	Disadvantage: * Expensive * Greater Installation effort	* Gigabit ethernet, 10G Ethernet (100m)
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantage: * High bandwidth * Immune to interference * Low loss bandwidth * versatile Disadvantage: * Limited distance * Cost * Size in bulky	* Speed of signal is 500m * Television network * High speed internet connection
Fibre optics cable	Single mode, Multi mode	100 Gbps	Advantage: * High speed * High bandwidth * High security * long distance Disadvantage. * Expensive. * Required skilled installa	* Maximum distance of fibre optics cable is around 100 mile

Result:

Thus, the study of different types of network cable successfully ~~cable~~ studied successfully.

Aaw 4/8/22

Exp No: 04-08-2026 Date:
03

Cisco Packet Tracer

Aim:

To study the Cisco packet Tracer simulation tool, understand its user interface, design a simple network and analyze the behaviour of Hub, Switch and peer to peer.

Cisco Packet Tracer:

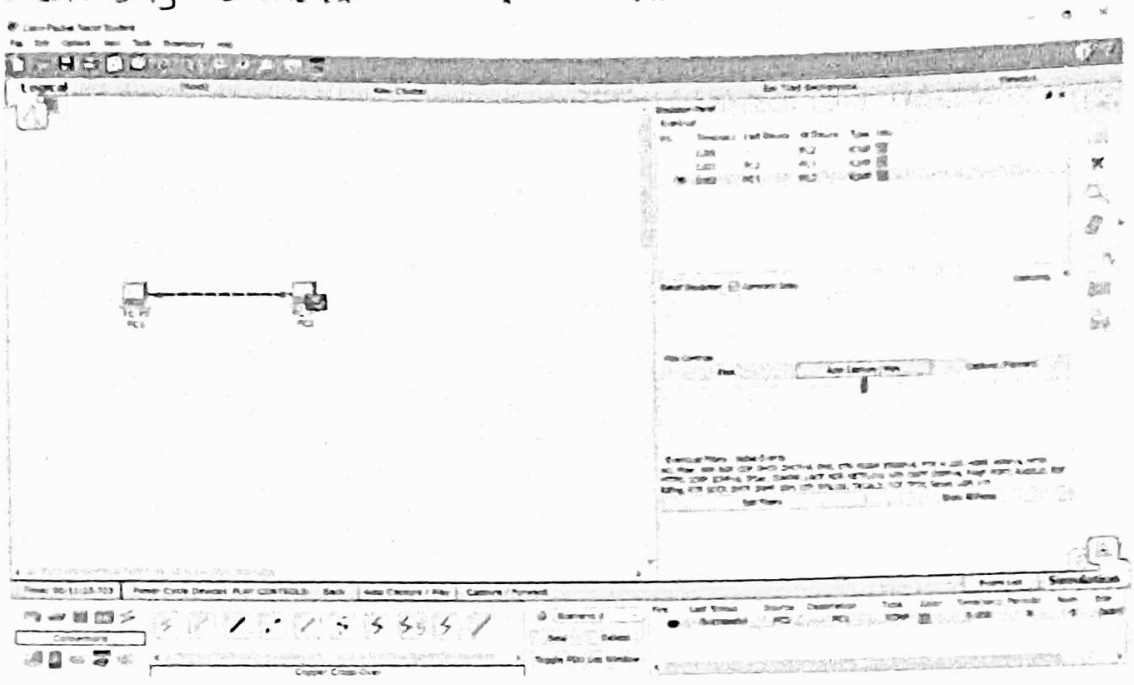
Cisco packet Tracer is a network simulation and modeling tool. It allows us to create and test computer networks without using real networking equipment. We can add PCs, switches, hubs, routers and other devices and observe how packets travel between them.

① peer-to-peer.

* Two Computers communicate directly.

* A peer to peer network was created by connecting 2 PCs directly. IP addresses were assigned to both PCs in the same network. The connectivity between the 2 PCs was tested using the ping command. Successful replies were received, confirming

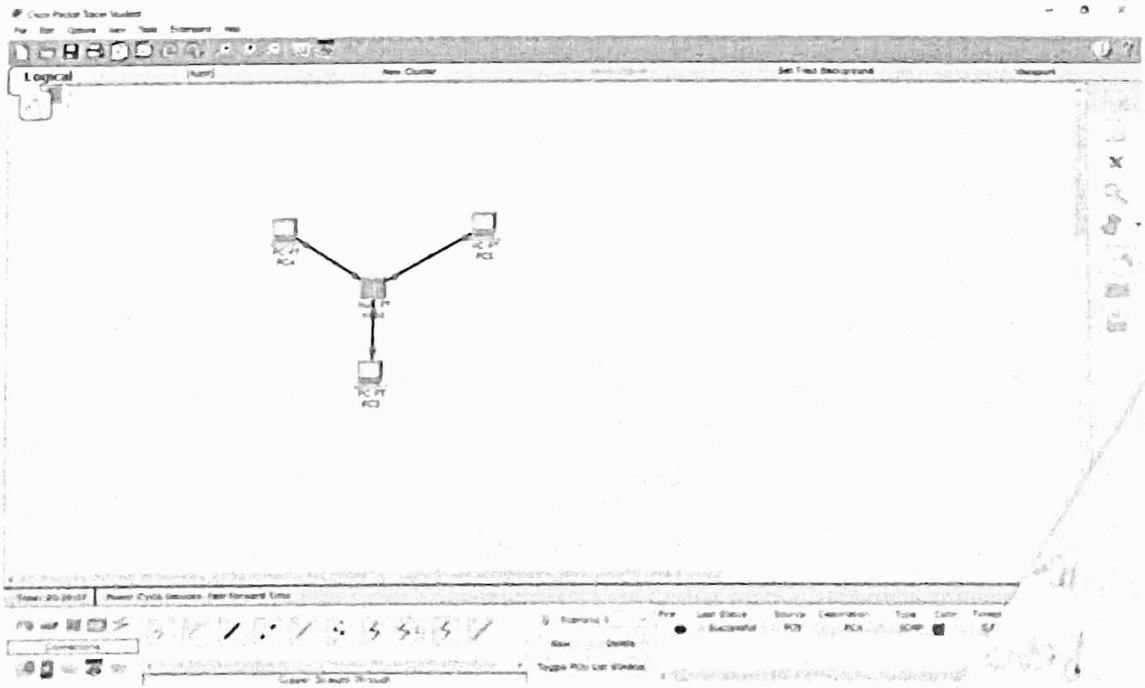
that the two PCs could communicate directly with each other.



② Network using Hub:

* Connected multiple computers & forwards received data to all ports.

* A Network was created by connecting ~~four~~ 3 PCs to a Hub using copper straigh through cables. IP addresses were assigned to all the PCs When ~~data was~~ sent from one PC to another, ~~the~~ Hub forwarded the packet to all connected ports. The destination PC received the packet successful communication was observed.



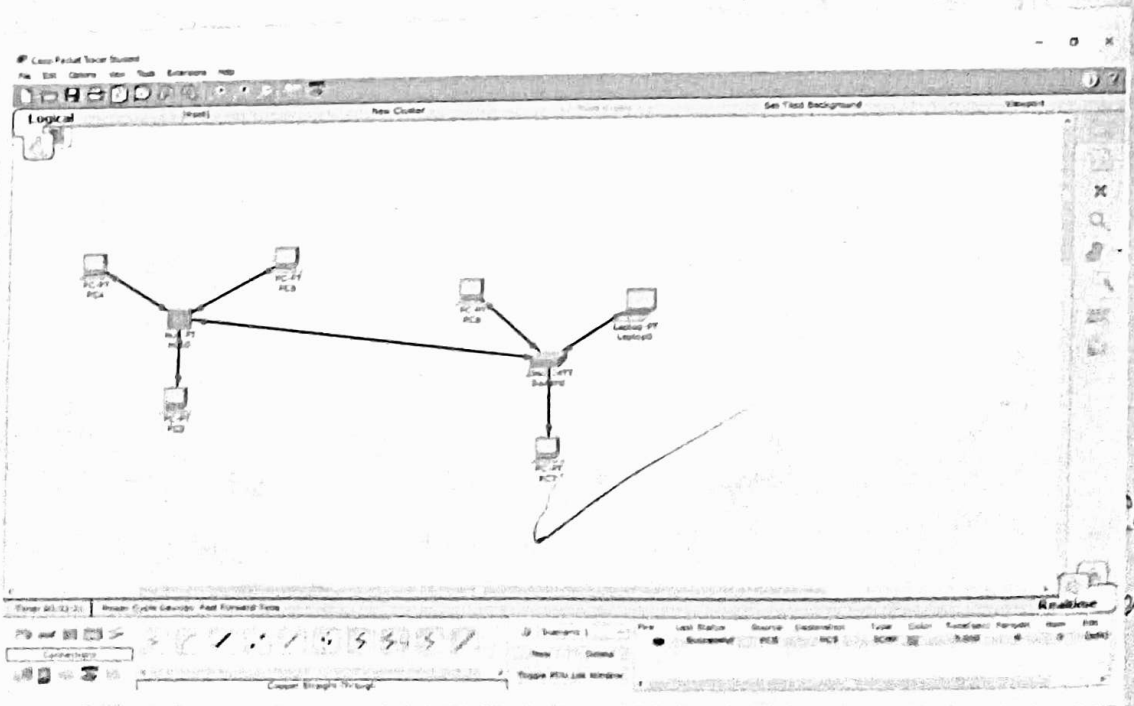
③ Networking Switch:

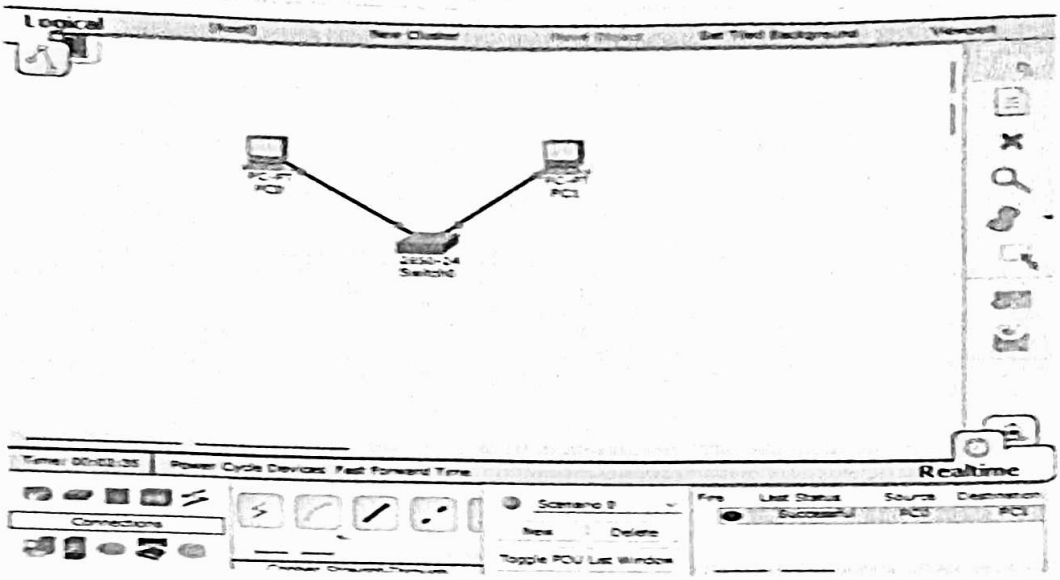
* Connects computers and forwards data toward the required destination.

* A network was created by connecting 3 pcs to a switch using copper straight through cables. IP addresses were assigned to all pcs and connectivity was tested using the ping command. The switch forwarded the packet to the appropriate destination port, allowing successful communication between the PCs

④ Hub and switch:

* Both Hub and switch were used in the network. The packet forwarding behaviour of both devices was observed, and compared. The hub forward packets to all connected ports. The switch forwards packets to the appropriate destination port. combination between pcs was tested using ping command





Result: 13/9/16

Thus, peer to peer, Hub & switch networks were successfully designed & their communication behaviour were observed using Cisco packet Tracer.

Expt. No: 04 Date: 10/8/26.

Setup and configure a LAN using a switch and ethernet cables in your lab.

Aim:

To setup and configure a LAN using a switch and ethernet cables in your lab.

What is LAN?

LAN stands for Local Area Network which is a group of computers and connected devices linked together in a small physical location like a school, home or office building.

How to set up LAN?

STEP 1: Connect the Hardware.

* place the router in the central location near a power outlet.
* Connect your modem to the routers WAN or internet port using a ethernet cable.

STEP 2: Connect your Devices.

Wired setup: Plug one end of an ethernet cable into your computers network port and the numbered LAN ports on the router.

STEP 3: Configure the Router.

* Type your IP address 192.168.1.1 into the address bar and press enter.

* Log In using the default username and password printed on the router.

STEP 4: Check the connectivity.

check the connectivity between switch and other machine by using PING COMMAND in the command prompt of the device.

STEP 5: confirm the connectivity.

Try to access the shared folder from other computers of the networks.

Result:

Wireless LAN using Switch and ethernet cables are executed successfully in CISCO PACKET TRACER.

10/8/20

Expt No: 5

Date: 18/8/26

Wire Shark

Aim:

To filter, capture, view, packet in wire shark tool.

Procedure

* Select local Area Connection in wireshalks.
* To capture Option.
* Select Stop capture automatically after 100 packets.
* Then click shark capture.
* Search TCP packets in search bar
* Save the packets.

① Create a filter to display only TCP/UDP packets, Inspect the packet and provide the flow graph.

* Select LAN connection in wireshark.
* To capture option.
* Select Stop capture automatically after 100 packets.
* Then click start capture.
* Save the packet.

(2) Create a filter to display only ARP packets and inspect the packets.

* Go to capture option.
* Select stop capture automatically after 100 packets.
* Click start capture.
* Search ARP packets in search bar.
* Save the packets.

(3) Create a filter to display only HTTP packets and inspect the packet.

* Select local Area Connection in wireshark
* Go to capture option.
* Select stop capture automatically after 100 packets.
* Then click start capture
* Search HTTP packets in search bar.

(4) Create a file to display only DNS packets and provide the flow graph.

* Go to capture option.
* Select stop capture automatically after 100 packets.
* Click start capture
* Search DNS packets in search bar.

* To use the flow graph click Statistics flow graph.

(5) Create a filter to display only DHCP packets and inspect the packets.

* Select local Area connection in wireshark.

* Go to capture option.

* Select stop capture automatically after 100 packets.

* Then click Start capture.

* Search DHCP packets in search bar.

* Save the packets.

Result:

Wireshark in packet capture tool experiment are studied and executed successfully.

Expt: No: 06
Date: 07/9/26

Error Detection & Error correction

Aim:

Write a program to implement error detection and correction using hamming code concept Make a test run to input data stream and verify error correction

Sender program.

* Input to sender file should be a text of any length program should convert the text to binary.
* Apply hamming code concept on the binary data and add redundant bit to it
* Save in a file called channel.

Code (sender.py).

```
data = list (map (int, input("enter data bit:"), strip()))
m = len (data)
r = 0
while (2**r) < (m+r+1):
    r+=1
n = m+r
hamming = [0]* (n+1)
j=0
for i in range (1, n+1):
```

```
        if (i(j-1)) != 0:
            hamming[i] = data[j]
            j += 1

for p in range(r):
    pos = 2**p
    parity = 0

for i in range(1, n+1):
    if i < pos:
        parity = hamming[i]

hamming[pos] = parity
code = hamming[1]
print("Data bits : data)
print("Hamming code : code)
```

output:

```
Enter data bits : 1011
Data bits : [1, 0, 1, 1]
Hamming code : [0, 1, 1, 0, 0, 1, 1]
```

Receiver code (receiver.py)

```
received = list(map(int, input("Enter received
                Hamming Code: ").strip()))

n = len(received)
r = 0
while (2**r) < n:
    r += 1
```

```
error pos = 0
for p in range(r):
pos = 2**p
parity = 0
 for i in range (1, n+1):
    if i & pos:
       parity ^= received[i-1]
    if parity != 0:
         error pos += pos

if error pos = 0;
   print ("No error detected").

else:
   print ("error detected at position:", error-pos)
   received [error - pos -1] ^= 1
   print ("corrected hamming code:", received)

data[]
for i in range (1, n+1):
    if (i & (i-1)) != 0:
    data.append(received[i-1])
    print ("original data list:", data)
```

Output:

0110011 change to 0111011
error detected at position: 4
corrected hamming code : [0,1,1,0,0,1,1]
original data bits : [1,0,1,1].

Result:

Thus. the hamming code algorithm was successfully implemented in python. the sender generated the hamming code from the given data. the receiver successfully detected and corrected a single bit errors.

Expt No: 07 Date: 08/9/26.

Flow control using sliding window protocol.

Aim:

write a program to implement flow control at data link layer using Sliding window protocol.

Sender:

* Get the window size from user
* Get the text message.
* Divide the message into frames.
* Send to SENDER_BUFFER
* Wait for Acknowledgement
* Read RECEIVER_BUFFER
* If ACK is received, send the next
* If NACK is recived, Retransmit the frame

Receiver:

* Read frames from SENDER_BUFFER
* Check the frame number
* If the frame number is expected, Send ACK
* Otherwise send NACK
* Write ACK/NACK into RECEIVER_BUFFER

Code (sender)

```
import time
w = int(input("window size: "))
msg = input("Message:")
i=0
while (i < lenmsg):
    end = min(i+w, len(msg))
    with open('Sender-Buffer', 'w') as f:
        for j in range(i, end):
            f.write(f'{i}: {msg[j]}\n')
            print("frame", j, "→", msg[j])
    time.sleep(2)
    with open('RECEIVER_BUFFER') as f:
        ack = f.read()
    i = int(ack[1])
print("Transimission completed")
```

Reciver.

```
import time
import os
expected = 0
while true:
    if not os.path.exists('Sender_Buffer'):
        continue.
```

```
with open ("Sender_Buffer") as f:
    frames = f.readlines()
for x in frames:
    n = int(x.split(":")[0])
    if n == expected:
        expected += 1
        print("Received frame", n)
        with open("Received Buffer",
                  "w") as f:
            f.write("ACK" + str
                    (expected))
    else:
        with open("Receive_Buffer", "w" as f:
            f.write("NACK" + str(expected))
        break
time.sleep(1)
```

Result:

The SLIDING WINDOW PROTOCOL was implemented successfully. Ack / NACK was simulated successfully.

28/7/21

Expt: NO:08
22/9/26

NMAP to Discover live hosts using Nmap scans on the tryhackme platform.

Aim:

To Discover live Hosts using NMAP Scans (ARP, ICMP, TCP/UDP) on the TryHackMe platform.

Room link: https://tryhackme.com/room/nmapsol.

Procedure:

TASK 1

* ARP scan: This scan uses ARP request to discover live hosts.

* ICMP Scan: This scan uses ICMP request to identify live hosts.

* TCP/UDP live PING Scan: This scan sends packets to TCP ports and UDP ports to determine live hosts

TASK 2 - Subnetworks

Network segment : It is a collection of computers linked together through a common "medium".

Subnetwork : It is usually the equivalent of one or more segments linked together and configure to utilize the same router in an IP network

IP network.

* Subnets with /16
* Subnets with /24

TASK 3 - Enumerating Targets.

We need to identify the targets we want to scan before we conduct any 'action'

① List

<machine_IP> <scanme.nmap.org>

<example> → will scan 3 IP addresses

② Range.

→ 10.11.12.15 -20

→ will scan 6 IP addresses

⑤ Subnet

⇒ <machine_IP> /30

⇒ will scan 4 IP addresses

Result:

Using Tryhackme platform, we learned and executed to discover live hosts on NMAP scan (ARP, ICMP, TCP/UDP)

EXPT NO: 09
28/4/26

Implementation of subnetting in cisco packet Tracer simulator.

Aim.

To implement classless IP subnetting using CISCO PACKET TRACER and verify connectivity between different subnets.

Procedure.

① open cisco packet Tracer and create a new network topology.

② Add routers, switches and PCs to the workspace.

③ connect the devices using appropriate cables.

④ Divide the network 192.168.1.0/24 into smaller subnets using /27 subnet mask.

⑤ configure the IP address and subnet mask for the router interface.

⑥ configure the PC's with suitable IP address, subnet masks and default gateways.

⑦ configure the switch ports and router interfaces.

⑧ use the PING command to test connectivity between the devices.

⑨ Verify that the packets are successfully transmitted between the connected networks

Advantages:

* Efficient use of IP address.
* Reduce Network congestion.
* Improves network management

IP addressing

Network	Subnet Mask	Router IP	PC IP Range
192.168.1.0 /27	255.255.255.224	192.168.1.1	192.168.1.11-15
192.168.2.0 /27	255.255.255.224	192.168.2.1	192.168.2.11-15
192.168.3.0 /27	255.255.255.224	192.168.3.1	192.168.3.11-15
192.168.4.0 /27	255.255.255.224	192.168.4.1	192.168.3.11-15

Result

The classless IP subnetting was successfully implemented using CISCO PACKET TRACER.

28/12/22

Expt No:12 End to End communication at Transport layer.
29/9/26

a) Implement ECHO client server using TCP Sockets.

Aim.

To implement an ECHO SERVER application using TCP sockets

Code: (ECHO SERVER)

```
import socket
S= Socket.Socket()
S.bind(("localhost", 5000))
S.listen()

c, addr = S.accept()
msg = c.recv(1024).decode()
print("client:", msg)
c.send(msg.encode())
c.close()
S.close()
```

ECHO CLIENT.

```
import socket
S = Socket.Socket()
S.connect(("localhost", 5000))
msg = Input("Enter message:")
```

```
s.send(msg.encode())
print("Echo:", s.recv(1024).decode())
s.close()
```

Input:

Enter message: Hello Server.

output:

```
Server.
client connected
Received: Hello server.
client:
Echo from server: Hello server.
```

Result:

The ECHO client server using TCP socket was studied and executed successfully.

29/9/22

b) Implement chat client server using TCP / IP sockets.

Aim:

To implement a chat client server application using TCP sockets.

Code. (SERVER)

```
import socket
s = socket.socket()
s.bind (("local host", 5000))
s.listen(1)
c, a = s.accept()
while True:
    msg = c.recv(1024).decode()
    print("client:", msg)
    if msg == "bye":
        break
    reply = input("server:")
    c.send(reply.encode())
c.close()
```

Client.

```
import socket
s = socket.socket()
s.connect(("localhost", 5000))
while True:
    msg = input("client: ")
    s.send(msg.encode())
    if msg == "bye":
        break
    print("server:", s.recv(1024).decode())

s.close()
```

Input.

client: Hello.
Server: Hi
client: Bye.

output.

client: Hello
server: Hi.

Result:

29/9/20 The chat client server applications were successfully studied and implemented using TCP sockets.

Expt. No: 13
30/9/26. Implement your own ping program

Aim:

To implement a ping program using python to check the connectivity between source and destination host.

Algorithm:

① Get the destination IP address from user.
② Send the request to destination
③ Wait for response.
④ Calculate the response time
⑤ Display whether the host is reachable.
⑥ Repeat the process for few Request

Code.

```
import os.
import time.
host = input ("enter host:")
for i in range(4):
    start = time.time()
    result = os.system("ping -n 1 "+
        host + " >null")
```

```
        end = time.time()
    if result == 0:
        print("Reply from", host,
"Time =", round((end-start)*
1000, 2), "ms")

    else:
        print("Request timed out")

    time.sleep(1)
```

Input:

Enter host: 127.0.0.1

output:

Reply from 127.0.0.1 Time = 1.23ms.

Reply from 127.0.0.1 Time = 0.98ms.

Result:

The ping program was successfully Studied and implemented.

30/9/22

Expt: NO:14

01/10/26. PACKET SNIFFING USING RAW SOCKET

Aim.

To write a program using Raw Socket to capture and display network packet

Algorithm.

① create a RAW socket

② Bind the socket to the network interface.

③ Receive packets from the network

④ Extract the IP header information.

⑤ Display the source IP, destination IP and protocol.

⑥ continue capturing packets.

Code.

```
import socket
S = socket.socket (socket.AF_INET, socket.
    SOCK_RAW, socket.IPPROTO_IP)

S.bind ("0.0.0.0", 0))
```

```
S. setsockopt (socket. IPPROTO_IP,
        Socket. IP_HDR INCL, 1)

Print ("packet sniffer started...")

While True:
    data, addr = S. recvfrom (65535)
    print ("packet received from:", addr[0])
```

Input

Network packets.

output

```
packet Sniffer started...
Packet received from: 192.168.1.5
Packet received from: 192.168.1.1
Packet received from: 142.120.183.14
```

Result:

The RAW SOCKET SNIFFING PROGRAM was successfully implemented and network packets was captured and displayed.